HONEYWELL SOFTWARE ENGINEERING TEAMS

AI Champions Programme

Module 04

Spec-Driven Development with GitHub Spec Kit / OpenSpec

Moving from prompt-only development to specification, architecture/plan, tasks, and acceptance criteria — driving precise, traceable implementation for greenfield features.

DURATION
2 Hours

FORMAT
Hands-On Lab

DELIVERED FOR
Honeywell Eng. Teams

How We'll Spend These Two Hours

Six connected moves — from why specifications matter to a hands-on spec-driven build.

01

From Prompt-Only to Spec-Driven Development

15 MIN

Why a persistent, structured specification changes what's possible in AI-assisted engineering.

02

Anatomy & Quality of a Good Specification

20 MIN

Requirements, constraints, interfaces/contracts, and acceptance criteria — and what makes each one "good enough."

03

The SDD Flow: Spec → Plan → Tasks → Implementation

25 MIN

How a specification becomes an architecture, a task list, and working, validated code.

04

Tooling the Workflow: GitHub Spec Kit / OpenSpec

15 MIN

The versioned artifacts — spec.md, plan.md, tasks.md — that carry the workflow end to end.

05

Traceability & Change Control

15 MIN

How every requirement stays linked to its task, implementation, and test — and what happens when a spec changes.

06

Hands-On Lab: Spec-Driven vs Prompt-Only

30 MIN

Build the same feature both ways and compare traceability, validation, and change resilience.

Module 04 in the 12-Module Journey

SDD is the first full engineering method this programme builds — everything since Module 01 has led here.

Why this matters: Module 05 applies this same spec → plan → tasks flow across a complete SDLC and into brownfield code; Module 06's test strategy is derived directly from the acceptance criteria written here.

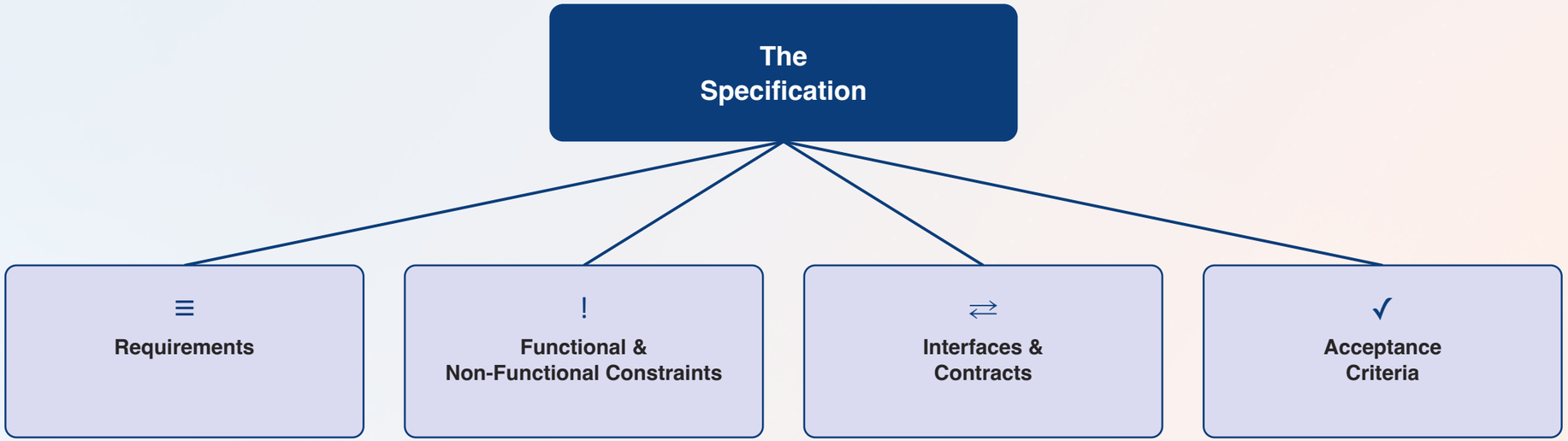
From Prompt-Only to Spec-Driven Development

The same shift Module 03 introduced for context, now applied to the whole feature.

PROMPT-ONLY	SPEC-DRIVEN
Requirements interpreted differently each time you ask	Requirements captured once, precisely, in a specification
No persistent artifact tying code back to intent	Every task traces back to the spec and its acceptance criteria
Acceptance criteria exist only in the requester's head	Acceptance criteria are explicit, written, and checkable
Any change means re-explaining the task from scratch	Changes flow through the spec — plan and tasks stay in sync

Anatomy of a Specification

Four parts, every one of them load-bearing for what comes next.



Why this matters: The plan is derived from requirements and constraints; the tasks are derived from the plan; and every task is only "done" when it satisfies its acceptance criteria. Weak input here weakens everything downstream.

What Makes a Specification "Good Enough"?

Four qualities the hands-on lab will check your specification against.

Unambiguous

One reasonable interpretation — no room for the model, or a teammate, to guess what was meant.

Testable

Every acceptance criterion can be checked mechanically or by clear, repeatable manual verification.

Traceable

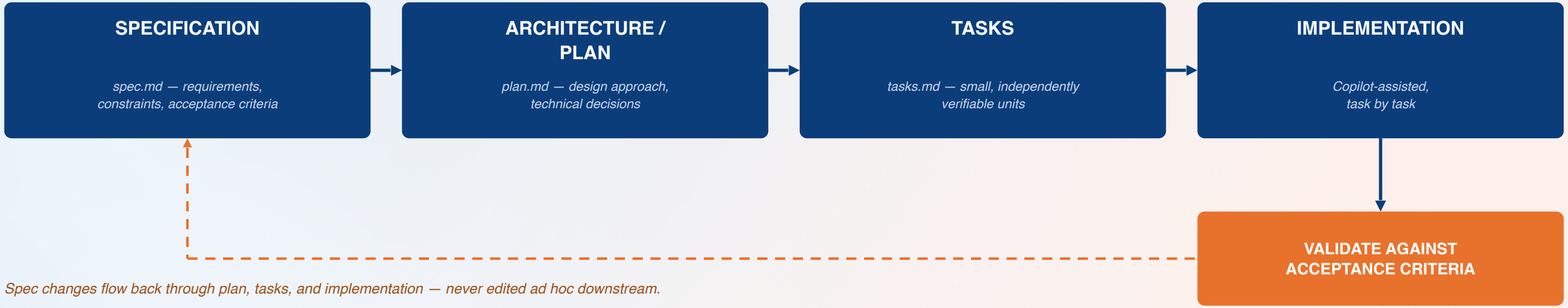
Each requirement maps to specific tasks, code, and tests — nothing gets implemented that isn't specified.

Complete

Functional and non-functional constraints are both captured — not just the happy path.

The SDD Flow: Spec → Plan → Tasks → Implementation

A forward path from intent to working code, closed by a feedback loop back to the specification.



Spec changes flow back through plan, tasks, and implementation — never edited ad hoc downstream.

The throughline: Nothing moves forward without what came before it, and nothing changes without flowing back through the spec — that discipline is what makes the output traceable, not just fast.

Tooling the Workflow: GitHub Spec Kit / OpenSpec

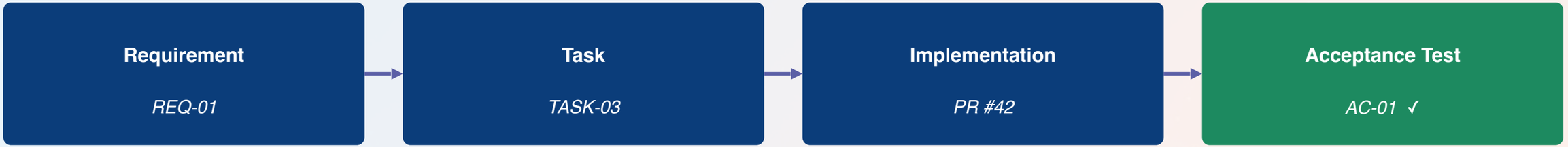
The same four-stage flow, viewed through the versioned artifacts that carry it.

spec.md	plan.md	tasks.md	Implementation + Validation
GitHub Spec Kit or an OpenSpec-equivalent pattern captures requirements, constraints, and acceptance criteria in one structured, versioned file.	Architecture and technical approach generated from the specification, reviewed by the team before implementation starts.	The plan broken into small, independently implementable and verifiable units of work.	Copilot implements task by task; each task's result is checked against the original spec's acceptance criteria.

The throughline: Every artifact is versioned alongside the code — the specification isn't a document that goes stale, it's part of the traceable record.

Traceability & Change Control

One requirement, followed all the way to a passing acceptance test.



Change control: When a requirement changes, the specification is updated first — the plan, tasks, and tests are then revised from that single source of truth, not edited ad hoc in code or in chat history.

Hands-On Lab Preview: Spec-Driven vs Prompt-Only

The 30-minute activity that closes this module — the same feature, built both ways.

Measure	Prompt-Only Baseline	Spec-Driven Approach
Requirements capture	Restated conversationally each time	Captured once in a versioned specification
Traceability	Not tracked	Requirement → task → implementation → test
Acceptance validation	Ad hoc, subjective review	Checked against explicit acceptance criteria
Handling a spec change	Re-explain the task from scratch	Update the spec; plan and tasks follow

Module 04 Outcomes & What's Next

- ✓ The shift from prompt-only to spec-driven development is understood
- ✓ The four parts of a specification are identified and drafted
- ✓ Specification quality — unambiguous, testable, traceable, complete — is checked
- ✓ The spec → plan → tasks → implementation flow is understood, loop included
- ✓ GitHub Spec Kit / OpenSpec artifacts are mapped to each stage
- ✓ A feature has been built spec-driven and compared to a prompt-only baseline

NEXT MODULE

Module 05 — Software Development: SDD to Complete Software SDLC (4 hours). Taking this same specification through the full SDLC, greenfield and brownfield, including architecture analysis and change-impact review.

Questions & Discussion

Module 04 — Spec-Driven Development with GitHub Spec Kit / OpenSpec